

```
In [9]: import matplotlib.pyplot as plt
import numpy as np
from numpy.linalg import svd
from sklearn import datasets
```

Problem 1

In this problem we will look at image compression using SVD, following the lines of the well-known "Eigenfaces" experiment. The basic concept is to represent an image (in grayscale) of size $m \times n$ as an $m \times n$ real matrix M . SVD is then applied to this matrix to obtain U , S , and V such that $M = USV^T$. Here U and V are the matrices whose columns are the left and right singular vectors respectively, and S is a diagonal $m \times n$ matrix consisting of the singular values of M . The number of non-zero singular values is the rank of M . By using just the largest k singular values (and corresponding left and right singular vectors), one obtains the best rank- k approximation to M .

The following code returns the dataset of 400 images.

```
In [10]: data = datasets.fetch_olivetti_faces()
images = data.images
```

(a) Given an $m \times n$ image M and its rank- k approximation A , we can measure the reconstruction error using mean ℓ_1 error:

$$\text{error}_{\ell_1}(M, A) = \frac{1}{mn} \|M - A\|_1 = \frac{1}{mn} \sum_{i=1}^m \sum_{j=1}^n |M_{i,j} - A_{i,j}|.$$

For $k = 1, \dots, 30$, take the average rank- k reconstruction error over all images in the dataset, and plot a curve of average reconstruction error as a function of k .

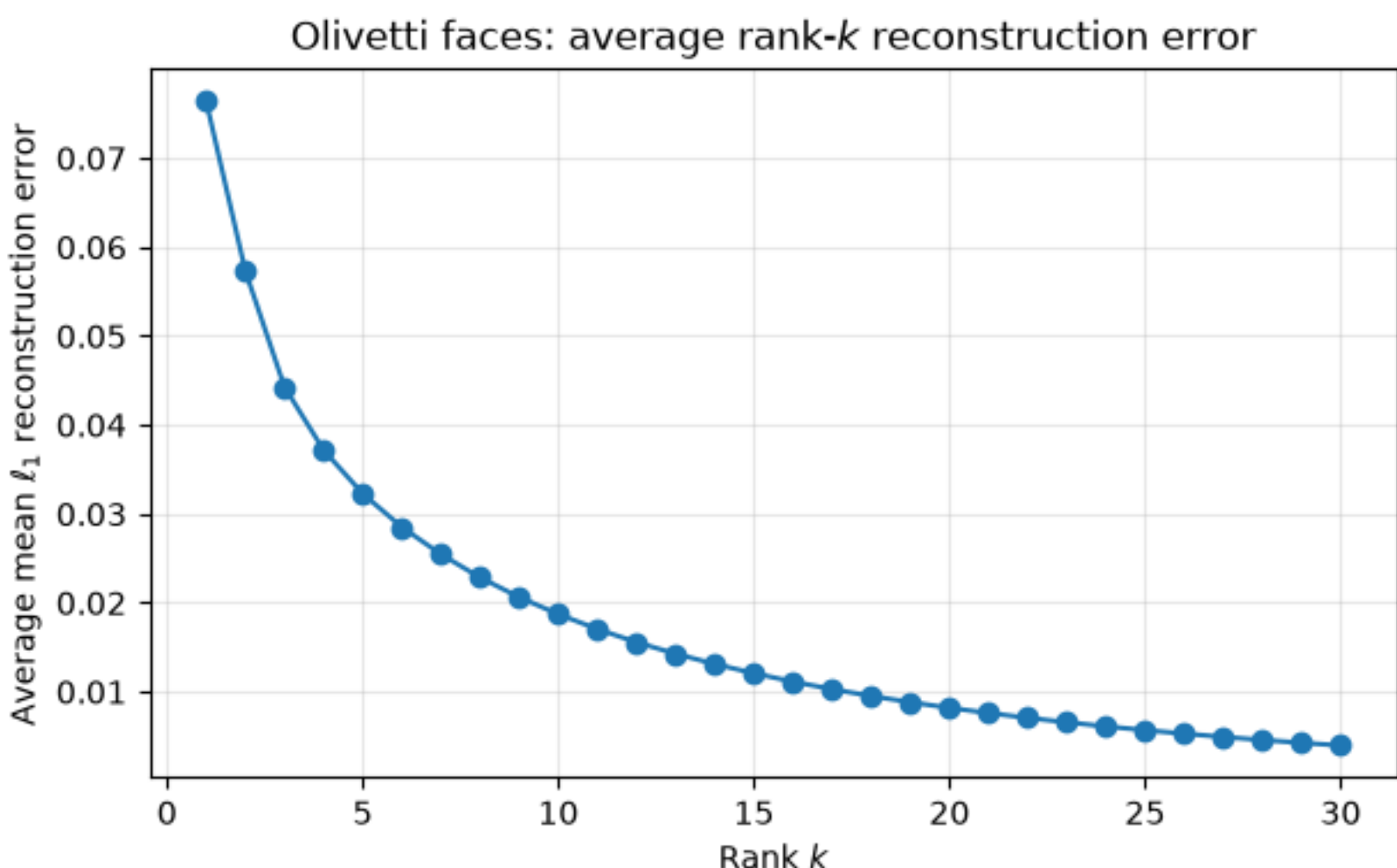
```
In [11]: # Returns the best rank-k approximation to M
def svd_reconstruct(M, k):
    # Advice: pass in full_matrices=False to svd to avoid dimensionality issues
    U, s, Vt = svd(M, full_matrices=False)
    return (U[:, :k] * s[:k]) @ Vt[:, k, :]
```

```
In [12]: def mean_l1_error(M, A):
return np.mean(np.abs(M - A))
```

```
In [13]: ks = np.arange(1, 31)
avg_errors = np.zeros(len(ks))

# Compute the SVD once per image and reuse it for every k
for img in images:
    U, s, Vt = svd(img, full_matrices=False)
    for i, k in enumerate(ks):
        A = (U[:, :k] * s[:k]) @ Vt[:, k, :]
        avg_errors[i] += mean_l1_error(img, A)
avg_errors /= len(images)

plt.figure(figsize=(7, 4))
plt.plot(ks, avg_errors, marker='o')
plt.xlabel('Rank $k$')
plt.ylabel(r'Average mean $\ell_1$ reconstruction error')
plt.title('Olivetti faces: average rank-$k$ reconstruction error')
plt.grid(alpha=0.3)
plt.show()
```



(b) Pick any image in the dataset, and display the following side-by-side as images: the original, and the best rank- k approximations for $k = 10, 20, 30, 40$. You will find the `imshow` method in matplotlib useful for this; pass in `cmap='gray'` to render in grayscale. Feel free to play around further.

```
In [14]: idx = 0 # any image in the dataset
M = images[idx]
ranks = [10, 20, 30, 40]

fig, axes = plt.subplots(1, len(ranks) + 1, figsize=(15, 3.5))
axes[0].imshow(M, cmap='gray')
axes[0].set_title('Original')
for ax, k in zip(axes[1:], ranks):
    A = svd_reconstruct(M, k)
    ax.imshow(A, cmap='gray')
    ax.set_title(f'k = {k}\nerror = {mean_l1_error(M, A):.4f}')
for ax in axes:
    ax.axis('off')
plt.tight_layout()
plt.show()
```



Problem 2

In this problem we visualize the Wisconsin breast cancer dataset in two dimensions using PCA. First, rescale the data so that every feature has mean 0 and standard deviation 1 across the various points in the dataset. You may find `sklearn.preprocessing.StandardScaler` useful for this. Next, compute the top two principal components of the dataset using PCA, and for every data point, compute its coordinates (i.e. projections) along these two principal components. You should do this in two ways:

1. By using SVD directly. Do not use any PCA built-ins.
2. By using `sklearn.decomposition.PCA`.

The two approaches should give exactly the same result, and this also acts as a check. (But note that the signs of the singular vectors may be flipped in the two approaches since singular vectors are only determined uniquely up to sign. If this happens, flip signs to make everything identical again.)

Your final goal is to make a scatterplot of the dataset in 2 dimensions, where the x-axis is the first principal component and the y-axis is the second. Color the points by their diagnosis (malignant or benign). Do this for both approaches. Your plots should be identical. Does the data look roughly separable already in 2 dimensions?

```
In [15]: cancer = datasets.load_breast_cancer()
```

```
In [16]: from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X, y = cancer.data, cancer.target # y: 0 = malignant, 1 = benign
X_std = StandardScaler().fit_transform(X) # mean 0, std 1 per feature

# Approach 1: PCA via SVD directly (no PCA built-ins)
U, s, Vt = svd(X_std, full_matrices=False)
pcs_svd = Vt[:2].copy()
proj_svd = X_std @ pcs_svd.T

# Approach 2: sklearn PCA
pca = PCA(n_components=2)
proj_pca = pca.fit_transform(X_std)

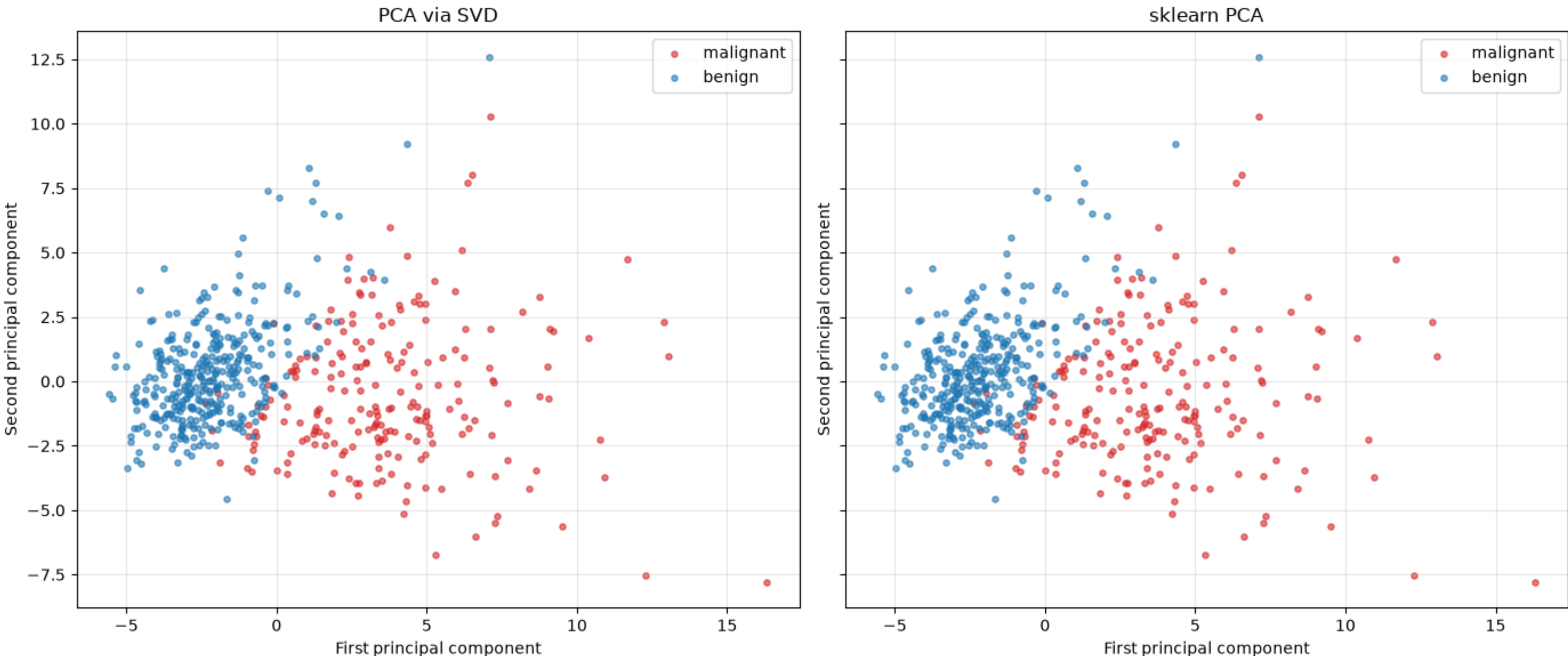
# Align signs (singular vectors are only unique up to sign)
for j in range(2):
    if np.dot(pcs_svd[j], pca.components_[j]) < 0:
        pcs_svd[j] *= -1
        proj_svd[:, j] *= -1

print('Projections identical:', np.allclose(proj_svd, proj_pca))
print('Explained variance ratio:', pca.explained_variance_ratio_)

fig, axes = plt.subplots(1, 2, figsize=(14, 6), sharex=True, sharey=True)
labels = {0: 'malignant', 1: 'benign'}
colors = {0: 'tab:red', 1: 'tab:blue'}

for ax, proj, title in zip(axes, (proj_svd, proj_pca), ('PCA via SVD', 'sklearn PCA')):
    for cls in (0, 1):
        mask = y == cls
        ax.scatter(proj[mask, 0], proj[mask, 1], s=14, alpha=0.6,
                  c=colors[cls], label=labels[cls])
    ax.set_xlabel('First principal component')
    ax.set_ylabel('Second principal component')
    ax.set_title(title)
    ax.grid(alpha=0.3)
    ax.legend()
plt.tight_layout()
plt.show()
```

Projections identical: True
Explained variance ratio: [0.44272026 0.18971182]



The two classes are mostly separated by the first two principal components, but there is still some overlap between them.